

Agenda

Backend LLD Module 2

SOLID

↳ Single Responsibility
→ Open / Close

2 classes
SOLID Principles

7 classes
Design Patterns

1 class
UML Diagram

10 DPs
23-24 DPs → GOF

↓
class + prp ↑

context

mock : Backend LLD 1

SOLID Principles

Principles? → guidelines

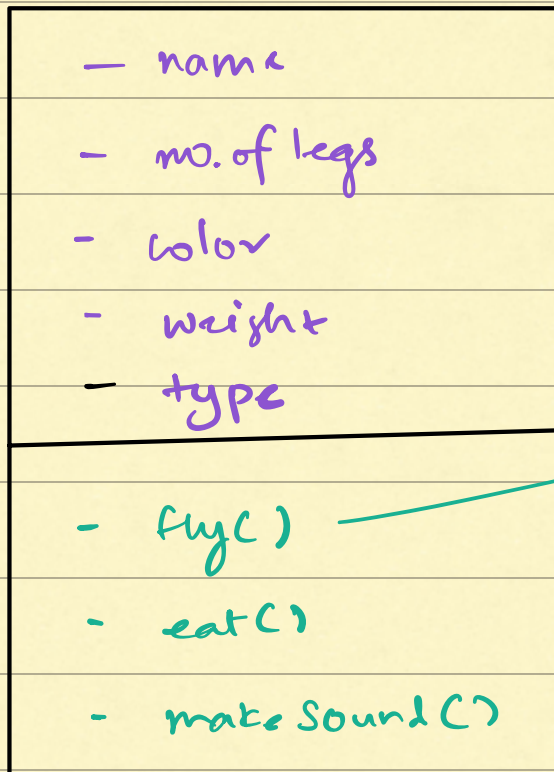
- S - Single Responsibility Principle
- O - Open / closed
- L - Liskov's substitution
- I - Interface Segregation
- D - Dependency Inversion

code → Maintainable
Readable
Understandable
Modular
Extendible
Reusable
Testable.

Question : Designing a software system
which records all kinds of
birds.

QD

Class Bird



fly() {

if (type == "Pigeon")

=====
printf("type == \"Eagle\")
=====
}

?

Bird b1 = new Bird();

b1.type = "Pigeon";

✓
This violates
⑤ of SOLID
↳ SRP

Bird b2 = new Bird();

b2.type = "Eagle";

b1.fly();

b2.fly();

Single Responsibility Principle

→ Each code unit must have
a single responsibility (task) to
perform.

There shouldn't be more than 1 reason
to change a code unit

↳ function / class / package

Common Cases of SRP Violation :

1. Multiple IF/Else : (Reason)

Exceptions → ① if/else are part of algorithm / business logic.

— checkLeapYear()

→ ② Input classes / functions

```
if ( input == 1 ) {  
    printMenu();  
} else if ( input == 2 ) {  
    connectDBC();  
}  
elseif (input == 3) {  
    display()  
}
```

Example : class Utility {

100 funcs

}

→ Not a good design

X SRP.

2. Monster functions / Classes (Reason).

→ Too many code lines

3. Utility Classes

Open Close Principle:

Your code should be open for extension
but closed for modification

```
func() {  
    if ( — ) {  
        ≡  
    }  
    else if ( — ) {  
        ≡  
    }  
    else if ( — ) {  
        ≡  
    }  
}
```

(vi) Abstract Class Bird

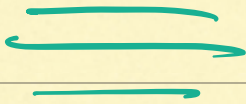
- name
- no. of legs
- color
- weight
- type
- fly() abstract
- eat()
- makeSound() abstract



class Pigeon extends Bird {

@Override

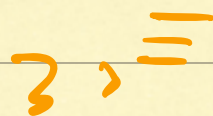
fly() {



}

class Crow extends Bird {

fly() {



class Eagle extends Bird {

{

fly() {



}

Bird b1 = new Pigeon();

b1.fly();

Bird b2 = new Eagle();

b2.fly();

VI

abstract class Bird

- properties
- fly() abstract.
- eat()

Pigeon {

fly() {

==

↓
Crow

↓
Eagle

Penguin extends Bird

{

@Override

fly() {

- throw Exception

- just print & do nothing

- empty Method.

}

}

function (Bird b) {

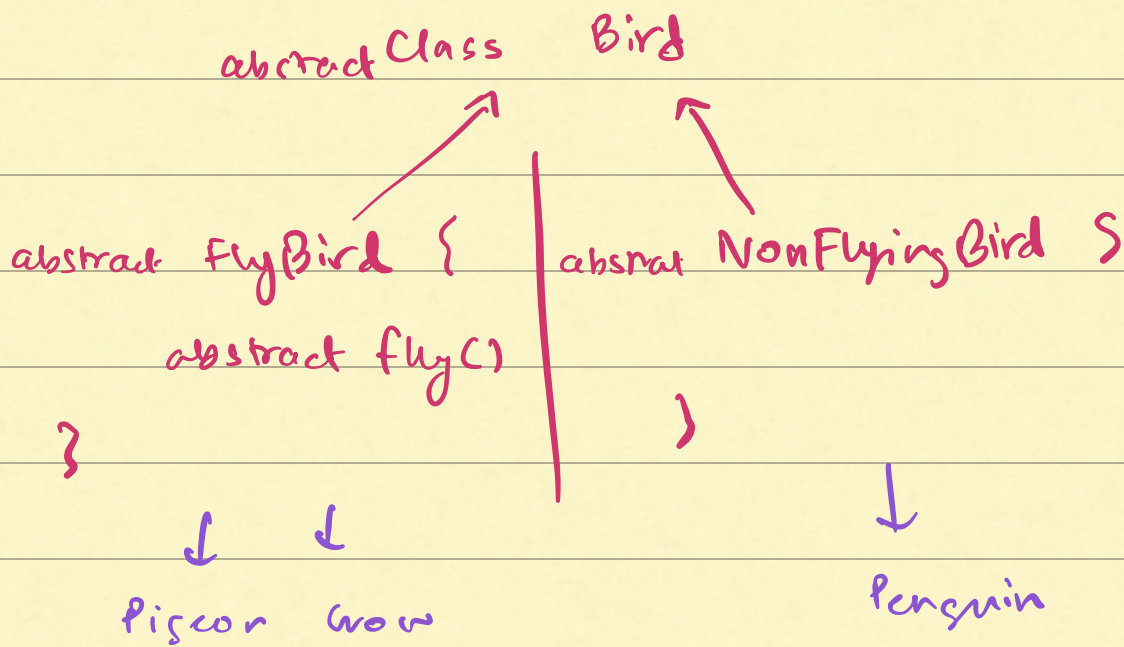
b.fly();

new Pigeon

new Eagle

X new Penguin

}



fly + swim

FlySwim Bird, FlyNonSwim Bird, NonFlySwim Bird,
Nonfly NonSwim Bird → 4

fly + swim + Run

→ 2^3 → 8

n features → 2^n class → Class Explosion

↳ Interface